Lumi Reading · Security & Privacy Evidence

T2 / T3

Patch & Vulnerability Management

Lumi Education Pty Ltd · ABN 45 700 349 015 · 24 July 2026

DRAFT

ST4S item: EV12 (patch management), also evidences T2 (centrally managed patching) and T3 (timely remediation of known-exploited vulnerabilities) **Version:** 0.1 DRAFT · **Date:** 2026-07-24 **Status:** Draft for review — not yet signed

1. Purpose and scope

This document describes how Lumi Reading learns about vulnerable dependencies, platforms and tooling; how it prioritises and applies patches; and how it verifies that a patch was applied and stays applied. Every automated control described is implemented in the repository and cited by file path. Where a deadline is a policy target rather than an evidenced-as-met fact, it is called out (see §9).

Scope: the Lumi monorepo (`lumi-ninc-au`, `australia-southeast1`) — the Flutter app (`lib/**`, `pubspec.lock`), Cloud Functions (`functions/**`), the privileged core (`packages/server-ops/**`), the two Next.js portals (`school-admin-web/`, `admin/`), the marketing site (`marketing-site/`), the CI workflows (`.github/workflows/**`), and the developer endpoint (the macOS build machine). It does **not** cover the managed platforms Lumi builds on — Firebase / Google Cloud, SendGrid, Twilio, the app stores — which are patched by the vendor and tracked through the vendor register, not this process.

2. Principles

- **Central management (T2).** Dependency updates are driven centrally by configuration in the repository (`.github/dependabot.yml`), not by ad-hoc manual bumps. Every update lands as a reviewed pull request through the same branch → PR → CI → squash-merge flow as any other change (see EV13 §3), so a patch is peer-reviewed and passes the full regression gate before it reaches `main`.
- **Detect, decide, verify.** Every advisory is triaged to a decision — patch, accept-with-rationale, or defer-and-track — recorded in the findings register, and (for applied patches) verified by re-running the audit that surfaced it.
- **Risk-based deadlines.** Remediation time is driven by severity and by whether the vulnerability is being actively exploited (§5).

3. Sources watched

Source	What it tells us	How it reaches us
GitHub security advisories / Dependabot alerts	Known-vulnerable versions of a dependency in our lockfiles	Dependabot alerts on the repo + weekly update PRs (`.github/dependabot.yml`)

Source	What it tells us	How it reaches us
`npm` / `pnpm` audit database	Advisories affecting the resolved dependency tree	Weekly `security-review.yml` job + per-PR release gate (§4.2)
OSV database (Google)	Cross-ecosystem advisories over npm **and** pub lockfiles	`sca-osv.yml`, SARIF → GitHub Security tab (§4.3)
Firebase / Google Cloud release notes	Runtime deprecations (e.g. Node 20 decommission), SDK security fixes	Manual review by the security lead; migrations tracked as work items
Flutter / Dart release notes	Pinned-toolchain security fixes (`flutter` is pinned to 3.44.6 in CI)	Manual review; pub updates arrive via Dependabot
Node.js release notes	Runtime EOL / security releases (functions run Node 22)	Manual review; CI pins Node 22
macOS / endpoint updates	OS and Xcode/CLI security updates on the build machine	macOS automatic security updates; formal device standard is pending (§9)

4. Automated tooling (cited)

All automation lives in `.github/`. Detail on triggers and enforcement is in EV13 §5; the patch-relevant behaviour is summarised here.

4.1 Dependabot — update automation (`.github/dependabot.yml`)

Weekly grouped update PRs for:

- **npm** in `/functions`, `/school-admin-web`, `/admin`, `/marketing-site` and the repo root (each capped at 5 open PRs, `minor` + `patch` grouped to reduce noise);
- **pub** at the root (the Flutter app);
- **github-actions** at the root (grouped as one).

Grouping means routine minor/patch bumps arrive as a small number of reviewable PRs; a major or security bump that Dependabot cannot group surfaces on its own.

4.2 Weekly dependency audit (`.github/workflows/security-review.yml`) — blocking

Scheduled Mondays 02:17 UTC (`cron: "17 2 * * 1"`) plus manual dispatch. Runs a **production** dependency audit on the security-relevant surfaces:

- Functions — `npm audit --omit=dev --audit-level=high`;
- Portals — `pnpm audit --prod --audit-level high`.

The same production audits are part of the per-release evidence checklist (`docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md`), so an accepted critical/high in a production dependency blocks a release unless a named, time-limited exception is recorded. This is the **blocking** dependency gate today.

4.3 OSV software-composition analysis (`.github/workflows/sca-osv.yml`)

`osv-scanner` (Google action v2) run **recursively over every lockfile** (npm + pub), uploading SARIF to the GitHub Security tab. Scheduled monthly (`cron: "17 3 1 * *"`) plus manual dispatch. **Report-only** (`continue-on-error`) while the findings register is triaged; the per-PR trigger is deliberately disabled pending

action-ref verification (documented inline in the workflow). This widens coverage beyond the prod-only, high-only weekly audit (it also sees pub and dev-tree advisories).

4.4 Secret scanning (`.github/workflows/secret-scan.yml`) — blocking

`gitleaks` v8.30.1 over the **complete git history** (`fetch-depth: 0`, `--redact=100`), on every push to `main`, **every** pull request, and on demand. This is the control that a patch PR (or any PR) does not introduce a leaked credential; the current history is gitleaks-clean.

5. Severity and remediation deadlines (T3)

Deadlines are measured from the date the advisory is triaged (first appears in an audit/scan or a Dependabot alert) to the date the fix is merged to `main`.

Class	Target time to patch
Actively exploited (known-exploited / in-the-wild, any severity)	**48 hours** — treat as an incident; if no clean patch exists, apply a mitigation/workaround (kill switch, config, WAF/rule change) within the window and track the permanent fix
Critical / High advisory affecting a production dependency	**14 days**
Moderate	Next scheduled dependency cycle (≤30 days), or with the next release touching that surface
Low / informational	Tracked in normal security work; no fixed SLA

Where no clean upstream fix exists within the deadline, the item is **not** silently allowed to lapse: it is recorded as *deferred-and-tracked* in the findings register with a written rationale, a compensating control where available, and the trigger that will close it (a patched upstream release). The worked example in §7 is exactly this case.

6. Verification — applied, and stays applied

A patch is only considered done when it is verified, and re-verified on an ongoing basis so it cannot silently regress:

- **Applied.** The dependency bump changes the committed lockfile (`package-lock.json` / `pnpm-lock.yaml` / `pubspec.lock`); the PR passes the full regression gate (`demo-readiness.yml`) before merge; the release evidence checklist confirms the production audit is clean (or the exception is recorded) before the change is deployed.
- **Stays applied.** The weekly `security-review.yml` audit (§4.2) and the monthly `sca-osv.yml` scan (§4.3) re-run against the current tree, so a regression (a transitive downgrade, a re-introduced vulnerable version) resurfaces as a fresh finding rather than being assumed fixed. Dependabot continues to alert on any newly-published advisory affecting the pinned versions.
- **Recorded.** The disposition of each advisory is entered in the patch register (§8) and, for security findings, in `docs/security/FINDINGS_REGISTER.md`.

7. Worked example — SCA-01 (`firebase-admin` transitive advisories)

The one open dependency finding demonstrates the deferred-and-tracked path.

- **Finding (SCA-01, `docs/security/FINDINGS_REGISTER.md`):** the Functions tree carries **9 transitive advisories (8 moderate, 1 high)** pulled in under `firebase-admin` — via `@google-cloud/firestore` → `google-gax`; `@google-cloud/storage` → `teeny-request` → `uuid`; and `retry-request`.
- **Decision:** *deferred-and-tracked.* There is **no clean upstream fix** — a forced upgrade (`npm audit fix --force`) breaks the build, because the vulnerable packages are pinned by `firebase-admin`'s own dependency ranges, not by Lumi's direct dependencies. Forcing them would diverge from a supported `firebase-admin` release.
- **Compensating position:** the advisories are transitive and none is flagged as actively exploited; the affected code paths are server-side (Admin SDK), not client-reachable.
- **Trigger to close:** bump `firebase-admin` when a release lands that resolves the ranges. Dependabot (§4.1) and `osv-scanner` (§4.3) both watch this tree, so the patched release will surface automatically as an update PR / cleared finding.
- **Register linkage:** tracked as SCA-01 (mapped to ST4S T2/T3/AP1/EV11) with a dated change-log entry.

8. Patch register (template)

The patch register is the running record of every advisory and its disposition. It is a superset view of the dependency rows already in the findings register, formatted for the T2/T3 evidence. Template:

Advisory (ID / link)	Package (+ path)	Severity	Exploited?	Decision (patch / accept / defer)	Applied date	Verified (date + how)	Notes / trigger to close
GHSA-... / OSV-...	`pkg@ver` (`/functions`)	High	No	Patch	2026-07-...	2026-07-... — `npm audit` clean + CI green	—
SCA-01 (register)	`firebase-admin` transitive (`/functions`)	High/Mod	No	Defer + track	—	Weekly audit + osv-scanner watching	Bump `firebase-admin` when patched release lands

Column notes: **Applied date** is the merge-to-`main` date; **Verified** records the audit/scan that confirmed the fix and the date it last passed; a *defer* row stays open with its close-trigger until the patched release lands.

9. Endpoint / OS patching

The developer/build endpoint (macOS) receives Apple's automatic security updates; Xcode and command-line toolchains are updated when a build requires or a security release warrants it. A **formal device standard (ST4S A10)** — the enumerated macOS security settings and update posture — is a separate document that is not yet drafted (kickoff backlog). Until it is signed, endpoint patching is asserted here only as "automatic OS security updates enabled"; the full standard must not be read as in force.

10. Supporting evidence index

Control	Evidence
Central update automation	`.github/dependabot.yml`
Weekly production dependency audit (blocking)	`.github/workflows/security-review.yml`
SCA over all lockfiles (npm + pub)	`.github/workflows/sca-osv.yml`
Secret scanning on every PR/push	`.github/workflows/secret-scan.yml`
Full regression gate that a patch PR must pass	`.github/workflows/demo-readiness.yml`, `scripts/demo-readiness-local.sh`
Release evidence checklist (audit must be clean)	`docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md`
Findings register (incl. SCA-01 disposition)	`docs/security/FINDINGS_REGISTER.md`
Vulnerability assessment (context)	`docs/security/VULNERABILITY_ASSESSMENT_REPORT_2026-07-24.md`

11. Known gaps (for the reviewer)

- **Deadlines vs practice.** §5 sets the 14-day (Critical/High) and 48-hour (actively-exploited) targets as policy. Confirm these are actually being **met and recorded** in practice — the register must show real applied/verified dates against real advisories, not just the SLA text.
- **Named owner.** This process names "the security lead" throughout; confirm the named human responsible for triaging advisories and driving patches within the deadlines is recorded (this is the same role as EV13 §11 / the monitoring plan's Security Lead — likely Nic / Director, to be confirmed and signed).
- **Register cadence.** Decide and record how often the patch register is reviewed and reconciled (proposed: weekly, aligned to the Monday `security-review.yml` run), and who signs off the review.
- **Report-only SCA.** `sca-osv.yml` is monthly and report-only with its per-PR trigger disabled (§4.3). The blocking dependency control today is the weekly/PR `npm/pnpm audit` (prod deps, high+). Re-enabling the per-PR SCA trigger and making it blocking is the intended end state; confirm whether monthly is sufficient in the interim.
- **Device standard (A10).** Endpoint/OS patching (§9) is asserted only as "automatic OS security updates on"; the formal macOS device standard is not yet drafted or signed.
- **Sign-off.** This process requires Nic's sign-off before it is the governing document.